

Reviewer Pack — Minimal Brauer Group Order and Circuit Monotone Width Correl...

Entry bfca8fcfad38 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 16:39:13 UTC

Minimal Brauer Group Order and Circuit Monotone Width Correlation via Modular Forms

Entry ID: bfca8fcfad38 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 16:39:13 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every d -regular graph G , the minimal order of its associated brauer group ($br(G)$) is linearly correlated with its circuit monotone width $w_{mon}(G)$, such that $br(G) = \Theta(w_{mon}(G))$. Moreover, if G is a complete graph on n vertices, then $br(G) \geq 2^{(n-1)}$.

Rationale (proposer's reasoning):

Modular forms provide a rich algebraic structure that may encode complexity-theoretic properties. The brauer group order measures the non-triviality of this structure. By relating it to circuit monotone width, we explore a new avenue for characterizing computational hardness.

Taxonomy category: MODULAR_FORMS_TO_CIRCUIT_MONOTONE_WIDTH_CORRELATION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ddd7d76eebf48acf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a d -regular graph G on n vertices, the conjecture is supported if the Pearson correlation coefficient between the minimal order of the brauer group ($br(G)$) and circuit monotone width ($w_{mon}(G)$) across 30 seeds is ≥ 0.8 , AND for complete graphs, $br(G)$ is $\geq 2^{(n-1)}$. The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - modular forms AND Boolean circuit complexity - brauer group order AND circuit monotone width - $\theta(w_{\text{mon}}(G))$ AND d-regular graph

Top relevant hits considered: - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order p^{aq} - [http://arxiv.org/abs/2402.06620v2] The Brauer groups of moduli of genus three curves, abelian threefolds and plane curves - [http://arxiv.org/abs/math/0301180v6] A generalization of the topological Brauer group

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    d = 3
    instances_tested = 0
    total_br = 0
    total_w_mon = 0
    n_max = 0

    for _ in range(30):
        # Generate a random d-regular graph on n vertices
        G = generate_d_regular_graph(n, d)
        if not G:
            continue

        instances_tested += 1
        n_max = max(n_max, n)

        # Compute the minimal order of the brauer group (br(G))
        br_G = compute_brauer_group_order(G)
        total_br += br_G

        # Calculate the circuit monotone width w_mon(G)
        w_mon_G = calculate_circuit_monotone_width(G)
        total_w_mon += w_mon_G

    if instances_tested < 30:
```

```

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

    # Compute the Pearson correlation coefficient
    mean_br = total_br / instances_tested
    mean_w_mon = total_w_mon / instances_tested
    numerator = sum((br_G - mean_br) * (w_mon_G - mean_w_mon) for br_G, w_mon_G in zip(br_values,
    denominator = math.sqrt(sum((br_G - mean_br)**2 for br_G in br_values)) * math.sqrt(sum((w_mon
    correlation_coefficient = numerator / denominator if denominator != 0 else 0

    # Check the lower bound on br(G) for complete graphs
    is_complete_graph = all(len(G[i]) == n-1 for i in range(n))
    conjecture_holds = correlation_coefficient >= 0.8 and (is_complete_graph or br_G >= 2**(n-1))

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

def generate_d_regular_graph(n, d):
    if (n * d) % 2 != 0:
        return None

    G = [[] for _ in range(n)]
    edges_added = set()

    while len(edges_added) < n * d // 2:
        u = random.randint(0, n-1)
        v = random.randint(0, n-1)

        if u == v or (u, v) in edges_added or (v, u) in edges_added:
            continue

        G[u].append(v)
        G[v].append(u)
        edges_added.add((u, v))

    return G

def compute_brauer_group_order(G):
    # Placeholder for the actual computation of the brauer group order
    # This is a dummy implementation and should be replaced with the actual algorithm
    return random.randint(1, 100)

def calculate_circuit_monotone_width(G):
    # Placeholder for the actual calculation of the circuit monotone width

```

```

# This is a dummy implementation and should be replaced with the actual algorithm
return random.randint(1, 100)

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_37ba01a7.py", line 110, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_37ba01a7.py", line 57, in run_trial
    numerator = sum((br_G - mean_br) * (w_mon_G - mean_w_mon) for br_G, w_mon_G in zip(br_values, w_mon_
                                                         ^^^^^^^^^
NameError: name 'br_values' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson correlation coefficient could not be calculated to verify the conjecture. | next: Investigate and fix the

error in the test code to ensure it can run to completion and produce the necessary data for calculating the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24503
2	preregistration	ollama_remote	glm4:latest	0	0	20200
3	novelty	ollama_remote	glm4:latest	0	0	18266
4	novelty	ollama_remote	glm4:latest	0	0	10348
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16683
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13527
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16009
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12582
9	judge	ollama_remote	glm4:latest	0	0	12651

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 144769 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/bfca8fcfad38.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bfca8fcfad38.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bfca8fcfad38.tar.gz (if generated)